CS 6515: Graduate Algorithms

Lecture 6: Graph Representations, BFS, Shortest Paths, and MSTs

Instructors: Sahil Singla and Gerandy Brito

Fall 2026

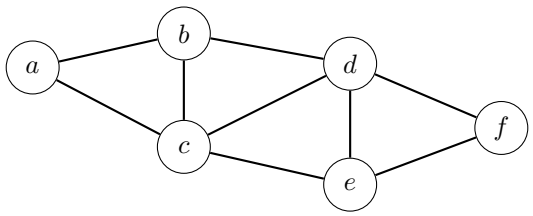
From the previous lecture. Dynamic programs can be viewed as dependency graphs whose vertices are subproblems. An MDP also has a transition graph, although an action may lead to a distribution over next states. We now make the graph itself the input and ask structural questions about reachability, shortest paths, and cheap ways to connect all vertices.

1 Announcements

1. HW1 graded. Submit any regrade requests very soon.
2. HW2 is due Sep 17.
3. Exam 1 is Sep 22.

2 Graphs and their representations

A graph is $G = (V, E)$, where V is a set of vertices and E is a set of edges. Write $n = |V|$ and $m = |E|$. Graphs may be directed or undirected, weighted or unweighted.



An undirected graph with $n = 6$ and $m = 9$. For example, $\deg(c) = 4$ and the neighbors of c are a, b, d, e.

For an undirected graph, the degree $\deg(v)$ is the number of edges incident to v, and

$$\sum_{v \in V} \deg(v) = 2m.$$

Representation	Space	Typical operation
Adjacency matrix	$\Theta(n^2)$	Test $(u, v) \in E$ in $\mathcal{O}(1)$
Adjacency lists	$\Theta(n + m)$	Scan all neighbors in $\Theta(\deg(u))$

Adjacency lists are the default for sparse-graph algorithms because scanning all lists costs $\Theta(n + m)$.

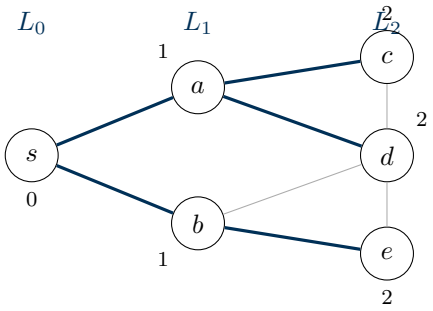
3 Breadth-first search

In an unweighted graph, the length of a path is its number of edges. Given a source s, breadth-first search (BFS) explores vertices in nondecreasing order of their shortest-path distance from s.

Algorithm. Initialize $\text{dist}[s] = 0$, set all other distances to $+\infty$, and place s in a first-in, first-out queue. Repeatedly remove a vertex u from the queue. For every neighbor v that has not been discovered, set

$$\text{dist}[v] = \text{dist}[u] + 1, \qquad \text{parent}[v] = u,$$

and append v to the queue.



BFS processes the distance layers L_0, L_1, L_2 in order. The blue edges are one possible parent tree; the gray edges do not shorten any layer.

Theorem (BFS correctness). When BFS finishes, $\text{dist}[v]$ is the minimum number of edges on an s-v path for every reachable vertex v. Parent pointers form a shortest-path tree.

Proof idea. The queue processes vertices one distance layer at a time. When v is first discovered from u, BFS has already found a path of length $\text{dist}[u] + 1$. If a shorter path to v existed, its preceding vertex would have belonged to an earlier layer and would have discovered v first.

Each vertex enters the queue once and each adjacency-list entry is scanned once, so BFS takes $\mathcal{O}(n + m)$ time.

Exercise. Run BFS from a chosen source on a graph containing a cycle. Label distance layers, draw the parent tree, and explain why non-tree edges do not invalidate the distances.

4 Nonnegative weighted shortest paths

BFS works because every edge contributes the same unit length. Now every edge e has a length $w(e) \geq 0$, and path length is the sum of its edge lengths.

4.1 A pseudo-polynomial reduction

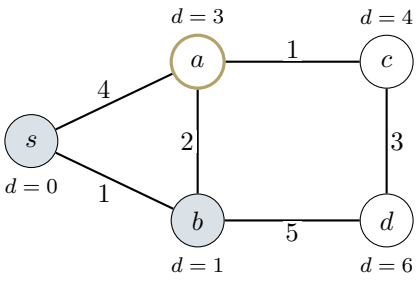
If all weights are positive integers, replace an edge of weight w by a path of w unit edges, introducing $w - 1$ new vertices. BFS in the expanded graph finds weighted shortest paths, but the graph size can depend on the numeric weights rather than their bit lengths. This is therefore only a pseudo-polynomial approach.



4.2 Dijkstra's algorithm

Dijkstra's algorithm maintains a tentative distance $d[v]$ and repeatedly settles the unsettled vertex of minimum tentative distance.

1. Set $d[s] = 0$ and $d[v] = +\infty$ for $v \neq s$. Store unsettled vertices in a min-priority queue keyed by $d[v]$.
2. Extract a vertex u of minimum key and mark it settled.
3. For every outgoing edge (u, v) with v unsettled, *relax* it: if
$$d[u] + w(u, v) < d[v],$$
update $d[v]$ and set parent$[v] = u$.
4. Repeat until the priority queue is empty.



After settling s and b, relaxing edges out of b gives $d[a] = 3$ and $d[d] = 6$. Vertex a is the next minimum-key vertex.

Theorem (Dijkstra correctness). If all edge weights are nonnegative, then whenever a vertex u is extracted, $d[u]$ equals the true shortest-path distance from s to u.

Proof idea. Suppose u is the first incorrectly settled vertex and consider a shortest path from s to u. Let (x, y) be the first edge on that path leaving the settled set. Vertex x was settled correctly, so relaxing (x, y) made $d[y]$ at most the length of the path prefix to y. Nonnegative weights imply that this prefix is no longer than the full path to u. Since u had minimum key, $d[u] \le d[y] \le \text{dist}(s, u)$. Every finite tentative distance is the length of an actual s–u path, so $d[u] \ge \text{dist}(s, u)$ as well. Hence $d[u] = \text{dist}(s, u)$.

With a binary heap, there are n extract-min operations and at most m decrease-key operations, for a running time of

$$\mathcal{O}((n + m) \log n).$$

4.3 Remarks: beyond the binary-heap bound

Fibonacci heaps. Dijkstra performs at most m decrease-key operations but only n extract-min operations. A Fibonacci heap supports decrease-key in $\mathcal{O}(1)$ amortized time and extract-min in $\mathcal{O}(\log n)$ amortized time. Thus, it improves the running time to

$$\mathcal{O}(m + n \log n).$$

This classic theoretical improvement is due to Fredman and Tarjan (1987); binary heaps are often simpler and faster in practice.

This is still an active area of research. For directed graphs with real nonnegative edge weights, a 2025 result of Duan, Mao, Mao, Shu, and Yin broke the long-standing "sorting barrier" with a deterministic

$$\mathcal{O}\Big(m \log^{2/3} n\Big)$$

algorithm in the comparison-addition model (paper).

Dijkstra's algorithm itself does not allow negative edges. Nevertheless, for directed graphs with integral, possibly negative edge weights and no reachable negative cycle, recent randomized algorithms have achieved near-linear time. A 2022 breakthrough gave $\mathcal{O}(m \log^8 n \log W)$ time, and a 2023 improvement reduced this to

$$\mathcal{O}\big(m \log^2 n \, \log(nW) \, \log\log n\big)\,,$$

where W bounds the magnitude of the most negative edge (2022 paper; 2023 improvement). These algorithms use ideas substantially beyond Dijkstra's algorithm.

Exercise. Give a small graph with a negative edge on which Dijkstra's algorithm can settle an incorrect distance. Which line of the proof uses nonnegativity?

5 Minimum spanning trees

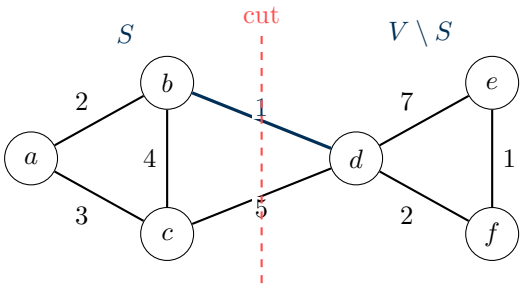
Shortest paths optimize routes from a source. A minimum spanning tree instead asks for one inexpensive backbone connecting all vertices. Let $G = (V, E)$ be a connected, undirected graph with nonnegative edge weights $w : E \to \mathbb{R}_{\ge 0}$.

Goal. Find $T \subseteq E$ that connects all vertices and minimizes

$$w(T) = \sum_{e \in T} w(e).$$

Observation. Some optimal solution is a tree. If a connected solution contains a cycle, remove a maximum-weight edge on that cycle; connectivity is preserved and the cost does not increase.

For $\varnothing \neq S \subsetneq V$, the cut $(S, V \setminus S)$ consists of all edges with one endpoint in S and the other outside S.



For the displayed cut, (b, d) is the lightest crossing edge. The cut property says that some MST contains it.

Theorem (cut property). Let e be a minimum-weight edge crossing any cut $(S, V \setminus S)$. There is a minimum spanning tree containing e. If e is the unique lightest crossing edge, every minimum spanning tree contains e.

Proof. Let T be an MST. If $e \in T$, we are done. Otherwise, adding e to T creates a cycle. Because the cycle crosses from S to $V \setminus S$ using e, it must cross back on some edge $f \in T$. By the choice of e, $w(e) \leq w(f)$. Thus

$$T' = T \cup \{e\} \setminus \{f\}$$

is a spanning tree with $w(T') \leq w(T)$. Since T was optimal, T' is an MST containing e. With a strict inequality against every other crossing edge, an MST omitting e would be improved, which is impossible. □

Exercise. Draw a weighted connected graph and three different cuts. Mark a light edge for each cut. Can two different light edges be certified simultaneously by the theorem?

Next time. The cut property immediately yields Prim's minimum-spanning-tree algorithm. We will then study matchings and their augmenting paths.